WhisperChain Final Threat Model

WhisperChain+ anonymous messaging platform | Finalized model of the current prototype | November 11, 2026

1 Scope and security position

This model covers registration, role assignment, blind credential issuance, round-based sending, encrypted inbox retrieval, recipient flagging, moderator freezing, and audit logging. It evaluates the current FastAPI, SQLite, and command-line implementation. Finalized means the model records the implemented controls and remaining risks; it does not mean all identified risks have been fixed. The highest priorities are transport security, anonymous credential revocation, and verifiable moderation decisions.

2 Assets and adversaries

Protected asset	Security property
Compliments and flagged reports	Confidentiality from other users and from operators without the relevant private keys; authenticity of the signed content.
Sender eligibility and anonymity	One accepted submission per approved credential per round without storing a direct issuance-to-pseudonym mapping.
Account and cryptographic secrets	Protection of passwords, sessions, user private keys, and server credential and audit signing keys.
Moderation state and audit history	Authorized freeze decisions, integrity of evidence, and detection of unauthorized history changes.
Service and routing metadata	Availability of delivery and moderation; limited exposure of recipients, pseudonym activity, and event timing.

Adversaries include unauthenticated outsiders, approved but abusive Senders, dishonest Recipients, malicious or compromised Moderators, compromised Admin accounts, network observers, database intruders, and attackers controlling the application host. A malicious operator may also issue credentials or manipulate public keys; the server is not assumed incapable of such actions.

3 Trust boundaries and data flow

Account boundary: client sends registration and login data; the service approves roles and issues an account session. **Issuance boundary:** an authenticated Sender requests a blind signature, then stores the unblinded credential locally. **Submission boundary:** the client sends recipient, anonymous token, and encrypted envelope without its account session. **Storage boundary:** the service writes encrypted messages and routing state to SQLite. **Disclosure boundary:** a Recipient re-encrypts signed content for moderators. **Audit boundary:** the service signs role-level events with a key stored separately from the database.

4 Assumptions and exclusions

We assume trusted client software and functioning cryptographic primitives for the baseline confidentiality argument. Client compromise, stolen keys, and server compromise remain explicit residual threats rather than being treated as impossible. HTTPS is a required deployment control, not an existing guarantee. Physical security and operating-system hardening are outside the implementation scope. Recipients can copy or disclose plaintext they legitimately decrypt; no cryptographic control can prevent that. Risk ratings on the following pages are qualitative priorities based on impact and attack feasibility, not measured probabilities.

5 Threat register for identity and authorization

Each entry separates an existing defense from its limits and the response still needed. High priority indicates a material confidentiality, eligibility, or moderation failure. Medium priority indicates a narrower exposure or an operational protection that remains incomplete.

ID and risk	Attack scenario	Implemented defense and limitation	Required response
T1 High	Network interception or impersonation	HTTP can expose passwords and sessions or allow directory substitution. Message encryption does not secure login or key lookup.	Default URL is HTTP. Require verified HTTPS outside loopback; test certificate rejection and absence of plaintext transport.
T2 High	Compromised Admin account	Approval and role endpoints are restricted to Admin. Admin has no message-reading privilege. However, reassignment or approval of attacker accounts can grant Moderator access to future reports or mint more Sender credentials.	Rejecting simultaneous Admin and Moderator roles is insufficient against role switching or collusion. Add independent approval and protected policy changes; audit privileged transitions.
T3 High	Revoked Sender retains sending ability	Account status is checked for sessions and credential issuance. Existing anonymous credentials have no account-status check during redemption.	Confirmed: revoked account submitted with HTTP 201. Add privacy-preserving revocation or short-lived credentials with carefully controlled renewal; verify revocation and role removal without adding identity to submissions.
T4 Medium	Shared or stolen credential and replay	Round signatures, credential verification, uniqueness constraints, and frozen-pseudonym checks reject ordinary forged or repeated tokens.	A complete pseudonym key and credential can be transferred. One slot remains per credential, not per human. Test concurrent replay and cross-round reuse; document sharing and device-theft limits.
T5 High	Moderator freezes an unrelated source	The normal client verifies reports before freezing, and the route requires Moderator. The service accepts a separately supplied pseudonym with no verified relation to the flag.	Confirmed: unrelated freeze target accepted with HTTP 200. Introduce verifiable evidence tied to the stored message and reject mismatched targets at the service boundary.
T6 Medium	Unauthorized inbox or flag access	Current-role checks and recipient ownership queries restrict inbox reads and flagging; unflagged content has no Moderator endpoint.	Retain checks on every path. Test all roles, missing sessions, other users' message identifiers, and future multi-role combinations; never rely on client menus for permission enforcement.

Code evidence: app/auth.py, main.py, tokens.py, messages.py, moderation.py; client.py. T3 and T5 were reproduced in an isolated database. The other entries combine source inspection with the existing test coverage; no claim of exhaustive penetration testing is made.

6 Threat register for privacy and integrity

ID and risk	Attack scenario	Implemented defense and limitation	Required response
T7 Medium	Recipient fabricates or substitutes evidence	A valid inner signature prevents inventing text under another uncompromised pseudonym. Reports are nevertheless not cryptographically bound to a particular stored envelope or message ID. The token signer and encrypted inner signer are not checked for equality.	Design a verifiable binding among token, envelope, message ID, report, and freeze target without exposing unflagged plaintext. Test replayed reports and mismatched inner and outer pseudonyms.
T8 High	Directory or server key substitution	Recipient fingerprints are pinned after first use. The first directory response remains trusted, moderator report keys are not pinned by the client, and the service supplies its own credential verification key.	Authenticate first-use keys through a trusted channel, pin appropriate server and moderator keys, and define approved rotation. Do not claim privacy against a malicious operator replacing keys.
T9 Medium	Traffic and pseudonym correlation	Blind issuance omits the pseudonym from the account issuance record. Anonymous sending omits bearer sessions, and fixed-size ciphertext reduces length leakage.	The service still sees a persistent pseudonym, recipient, source address, and timing. Document this anonymity boundary; minimize access logs and retention, and evaluate relays or batching if stronger anonymity is required.
T10 Medium	Database disclosure or ciphertext modification	Messages and reports are encrypted. AES-GCM detects envelope alteration and Ed25519 authenticates inner content. Passwords use salted scrypt; stored session tokens are hashed.	Recipient usernames and account records remain readable. Stolen user keys expose old ciphertext; no forward secrecy exists. Protect backups and key files, minimize metadata, and define key-loss response.
T11 High	Audit rewrite or signing-key compromise	Triggers reject ordinary edits and deletes; a hash chain and system signatures detect modified entries when the original verification key is trusted.	A privileged operator can drop triggers, truncate a valid suffix, or replace keys and history after host compromise. Publish independent signed checkpoints and protect signing keys; test edit and truncation detection separately.
T12 Medium	Flooding and identity farming	Admin approval and one message per credential per round limit accepted sends. Fixed envelope size and text limits constrain normal messages. Flag and freeze thresholds create audit alerts.	No general request throttling or registration identity proof is implemented. Attackers can flood rejected requests or exploit multiple approved accounts. Add bounded inputs, quotas, rate limits, and a documented enrollment policy.

Code evidence: whisper_crypto.py; client.py; app/db.py, blind.py, audit.py, messages.py, moderation.py, and tokens.py. The blind signature is a custom educational construction; cryptographic library use does not by itself validate the surrounding protocol.

7 Accountability and nonrepudiation

Sender signatures establish that a particular pseudonym key signed the recipient, round, and text. They do not establish a real person's identity. A shared or stolen key further weakens attribution. The audit log is signed by a system key, not separately by each Admin or Moderator. It supports role-level event integrity but cannot prove which individual performed a privileged action. The original draft's claim of individual privileged-action signatures must therefore be removed. If stronger accountability is needed, use independently verifiable signatures or privacy-preserving actor references with a clearly defined access policy.

8 Prioritized remediation and verification

Order	Required completion evidence
1 Transport and key authenticity	Run the deployed service over HTTPS and reject invalid certificates. Verify passwords and sessions are never sent over plaintext network transport. Establish and test trusted key fingerprints and approved key changes.
2 Anonymous eligibility	Demonstrate that account revocation and Sender-role removal prevent future sends from prior credentials. Preserve the absence of account identifiers on anonymous submissions and document the revised issuance or expiry rules.
3 Moderation evidence	Reject an arbitrary freeze target, a report copied to a different flag, and a token whose identity is inconsistent with the encrypted evidence. Ensure the proof mechanism does not reveal unflagged message content to the server.
4 Privileged access	Require independent authorization for sensitive role and key changes. Test an Admin switching roles or approving a controlled Moderator account; document the scope and limits of separation of duties.
5 Availability and audit durability	Test abusive request volume and enrollment quotas. Demonstrate detection of a deleted audit suffix through an independently stored checkpoint; exercise key protection and recovery.

9 Validation status and residual risk

The existing suite passed 38 tests covering the principal messaging, RBAC, cryptographic, moderation, and logging workflows. Two additional isolated checks confirmed T3 and T5. Passing tests are evidence for those specific checks, not a proof of anonymity, cryptographic protocol security, or deployment readiness. Add regression coverage for the confirmed failures and for the completion criteria above after changing the implementation.

Residual risks include recipient disclosure of decrypted text, credential sharing, endpoint compromise, cross-round pseudonym linkage, traffic correlation, lack of forward secrecy, dishonest enrollment, and collusion among privileged users. These must be stated during evaluation even after the immediate fixes. Stronger anonymity and individual accountability require additional design choices; neither is established merely by omitting usernames from the audit log.

10 Consistency and source record

Align the RBAC matrix with the implementation: the directory is public; unfreezing, configurable alert thresholds, key rotation and replacement, and second-person approval are not implemented. The completed application uses a command-line client rather than the browser client proposed in the draft. This model supersedes the draft's assumptions that TLS, general rate limiting, and individually signed privileged actions are already in place.

Sources: assignment PDF, WhisperChain+ Anonymous Messaging with Role-Based Control and Accountability; Week 1 Deliverables/Design Doc and Threat Model.md; RBAC Matrix - Sheet1.pdf; README.md; client.py; whisper_crypto.py; app/ modules; and tests/. Companion document: WhisperChain Security Design Report, November 11, 2026. Findings describe the local project reviewed on that date; proposed responses are not represented as implemented controls.